Advanced Databases CA2 – ERD – Relational Mapping – SQL CREATE and INSERT

Group Assignment – 25%

Due Date: Friday 1st May, 1pm
Submission Format: PDF (ERD image + SQL file + written answers)

👥 Group Details

- Group Number: 2
- Module Code: TU914Y2
- Lecturer: Farrah Higgins
- Student Names & IDs:
- Praise Eloho Amagada: C243550233
- Cian Dunne: C24505433

****No AI**In this assignment you are not allowed to use AI. By submitting this assignment, you are agreeing to complete the assignment entirely without any AI assistance.**

🧩 Assigned Mini-Case

📌 Domain Description

(Your group's unique scenario assigned in BrightSpace please paste your scenario below)

Write your assigned domain here:

Domain: University study groups

Entities: (student, course, lecturer, study group, meeting, room)

Constraints

Strong entities: student, course, lecturer, room

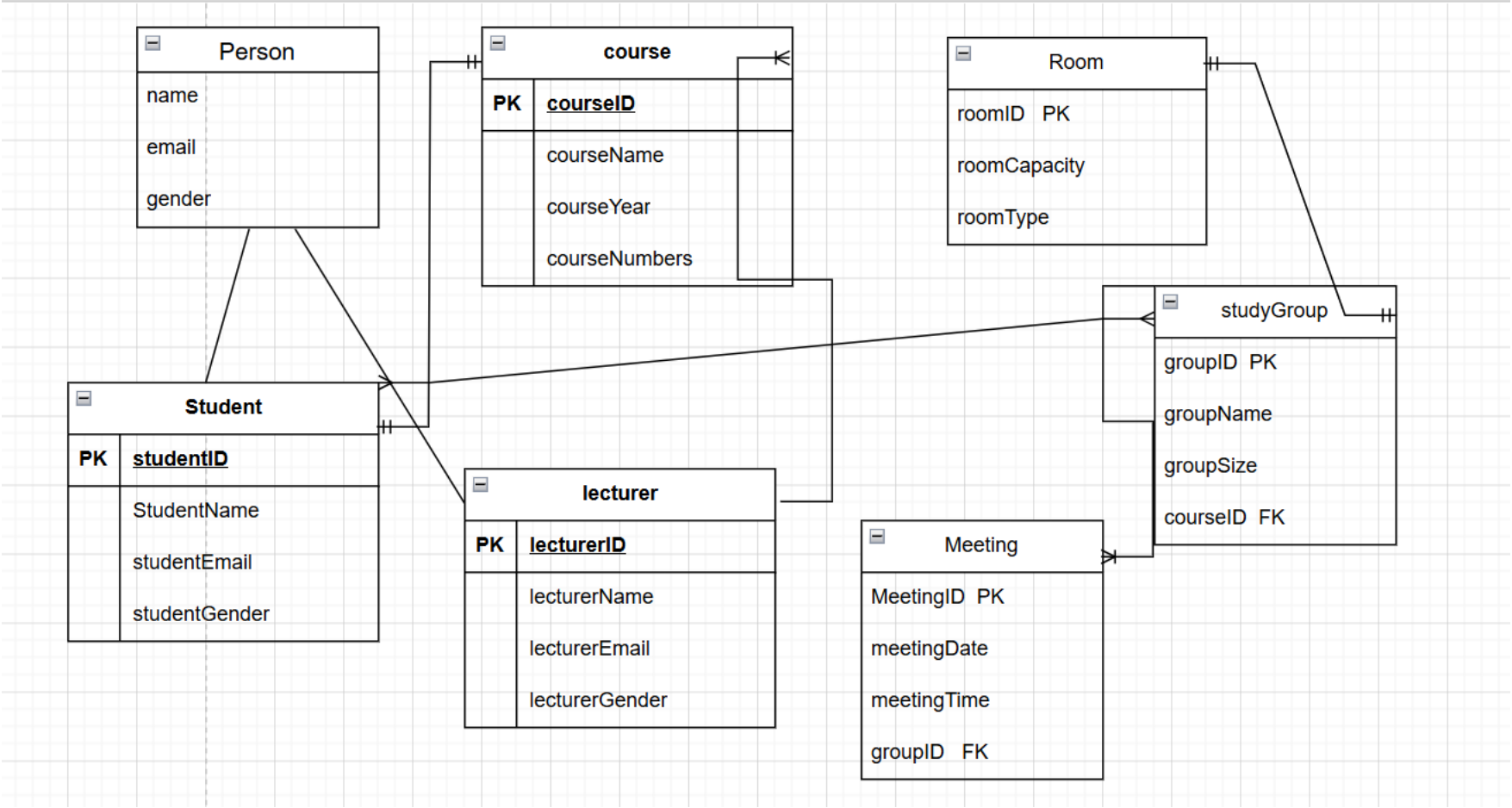
Weak entities: study group, meeting

Many to many relationships: **Student to StudyGroup**, a student can be in multiple study groups and a study group has multiple students

Business cases:

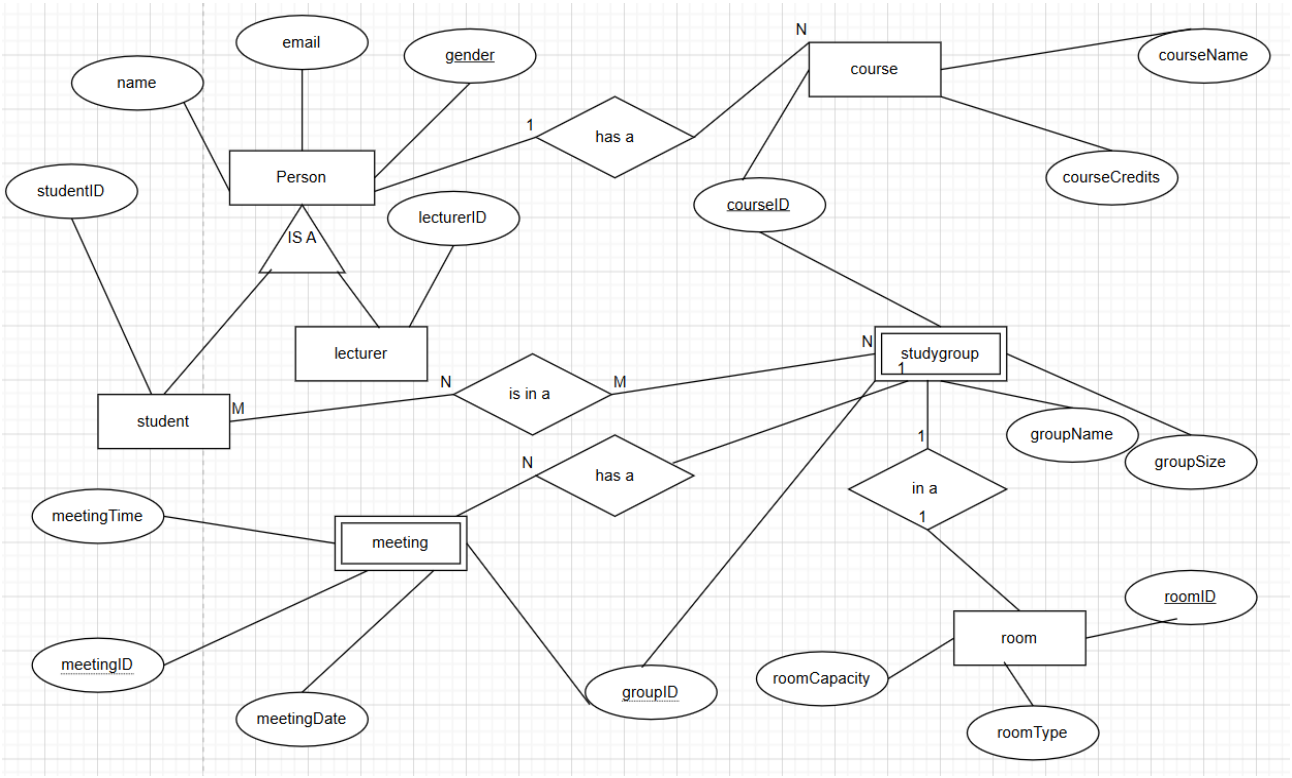
1) a study group must have a maximum of 6 students
2) a room can have a maximum of 1 study group

Part 1: ER Diagram (Crow's Foot Notation) (30%)



🧩 Part 2: Relational Schema (20%)

List all relations/tables created from your ERD, primary and foreign key, attributes with datatypes, many to many relationship, multivalued attributes, specialisation resolution and the correct order of tables to be created.



Part 3: SQL Implementation (30%)

A. CREATE TABLE Statements

Paste your SQL below:

```
-- CREATE TABLE statements
CREATE TABLE Course (
   courseID INTEGER,
   courseName TEXT,
   courseYear TEXT,
   courseNumbers TEXT,
   PRIMARY KEY (courseID)
);
CREATE TABLE Lecturer (
   lecturerID INTEGER,
   lecturerName TEXT,
   lecturerEmail TEXT,
   lecturerGender TEXT,
```

```
    PRIMARY KEY (lecturerID)
);
CREATE TABLE Meeting (
    meetingID INTEGER,
    meetingDate DATETIME,
    meetingTime DATETIME,
    groupID INTEGER,
    PRIMARY KEY (meetingID)
);
CREATE TABLE Room (
    roomID INTEGER,
    roomCapacity INTEGER,
    roomType TEXT,
    PRIMARY KEY (roomID)
);
CREATE TABLE Person (
    personID INTEGER,
    name TEXT,
    gender TEXT,
    email TEXT,
    PRIMARY KEY (personID)
);
CREATE TABLE Student (
    studentID INTEGER,
    studentName TEXT,
    studentEmail TEXT,
    studentGender TEXT,
    PRIMARY KEY (studentID)
```

```
);

CREATE TABLE StudyGroup (
  groupID INTEGER,
  groupName TEXT,
  groupSize INTEGER,
  courseID INTEGER,
  PRIMARY KEY (groupID)
);
```

B. INSERT Statements

Insert sample data:

- Minimum **3 records per main table**
- Ensure all foreign keys are valid
- Ensure constraints are respected

-- INSERT statements

SQL values used:

Course:

- INSERT INTO Course VALUES (1, 'Business Computing', 'Year 2', 'TU914');
- INSERT INTO Course VALUES (2, 'Software Development', 'Year 2', 'SD202');
- INSERT INTO Course VALUES (3, 'Data Analytics', 'Year 3', 'DA303');

Lecturer:

- INSERT INTO Lecturer VALUES (1, 'John Quad', 'john@email.com', 'Male');
- INSERT INTO Lecturer VALUES (2, 'Jessica Jones', 'jones@email.com', 'Female');
- INSERT INTO Lecturer VALUES (3, 'Mike Ross', 'mike@email.com', 'Male');

Room:

- INSERT INTO Room VALUES (1, 30, 'Lecture');
- INSERT INTO Room VALUES (2, 20, 'Lab');
- INSERT INTO Room VALUES (3, 50, 'Lecture');

studyGroup:

- INSERT INTO StudyGroup VALUES (1, 'Group A', 10, 1);
- INSERT INTO StudyGroup VALUES (2, 'Group B', 12, 1);
- INSERT INTO StudyGroup VALUES (3, 'Group C', 8, 2);

Meeting:

- INSERT INTO Meeting VALUES (1, #2026-05-01#, #10:00#, 1);
- INSERT INTO Meeting VALUES (2, #2026-05-02#, #12:00#, 2);
- INSERT INTO Meeting VALUES (3, #2026-05-03#, #14:00#, 3);

Person:

- INSERT INTO Person VALUES (1, 'Alice Green', 'Female', 'alice@email.com');
- INSERT INTO Person VALUES (2, 'Bob Marley', 'Male', 'bob@email.com');
- INSERT INTO Person VALUES (3, 'Emma Stone', 'Female', 'emma@email.com');

STudent:

- INSERT INTO Student VALUES (1, 'Alice Green', 'alice@email.com', 'Female');
- INSERT INTO Student VALUES (2, 'Bob Marley', 'bob@email.com', 'Male');
- INSERT INTO Student VALUES (3, 'Emma Stone', 'emma@email.com', 'Female');

Part 4: Reflection (20%)

Answer ALL questions clearly and in full sentences.

---

1. Most Difficult Part

What was the most difficult part of your design and why?

The most difficult part of our design was the making of the relational schema and the ER diagrams. They are practically the blueprints for our design and helps us implement our design into SQL. Things like figuring out the type of relationships(cardinality) our entities would between each other. Whether they would be many to many, 1 to many etc. Another difficult part was figuring out if there was another weak entity. StudyGroup was fairly straightforward to identify as a weak entity, but we weren't sure about room and meeting. In the end we decided meeting would be the weak entity between the two because there would not be a meeting if no room is booked whereas a room will always be there regardless of if there are students or not in it.

2. Alternative Design

Describe one alternative design you considered but rejected.

The only change to our approach we heavily considered was having a balance of strong and weak entities meaning we'd have 3 strong entities and 3 weak entities but ultimately decided against it and thought the 2 weak enities of studyGroup and meeting along with the 3 other strong entities

3. SQL Constraint Limitation

Identify one constraint that could NOT be enforced directly in SQL.

One constraint faced by using sql was similar to the business rule, in the way that a room could only be booked by one group at a time, meaning that meetings cant be overridden by a person trying to book a time already taken by another user.

4. Scalability

If your system scaled to 1 million users, what would break first and why?

If the system scalled to 1 million users what would break is most likely the Student and Person entities as if the database was required to store the data email, name, studentID etc for one million users it would become very large and begin to slow down significantly due to processing high amounts of data